Finch: Local-first Paper Reading

Finch contributors | Engineering demonstration | Revision 1

ABSTRACT

A local paper reader should recover attention as carefully as it recovers a file. This original demonstration describes a compact architecture for responsive PDF reading: prioritize the visible page, preserve a stable reading anchor, account for caches, and start optional capabilities only when requested. It is a reading fixture, not a report of measured performance.

1.1 A reader that remembers

A paper reader has a small but demanding task: put the right page in front of a person, keep it readable, and remember the place where work stopped. Finch treats this moment as the center of the application. A local document can be useful before a catalog has loaded or a plugin has been discovered.

Opening a document is a sequence of visible commitments. The window should respond first. The current page follows. Details that do not change the next reading action can arrive later. This ordering makes a slow document understandable without pretending that every operation is instantaneous.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 1. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Explore the wide-page section (page 5)

Illustrative external link: example.org

Restoring a reading position

Coordinates survive a changing window

2.1 Restoring a reading position

A saved page number is only a coarse location. A reader may stop halfway through an equation or between two columns. Store a point on the page and the place where that point appears in the viewport. These two facts describe the reading position even when the window is resized.

A restoration record also carries the document identity, its file revision, and a zoom mode. Explicit scale and fit-to-width are different intentions. A change in display density should preserve the intended content size while leaving the page anchor stable.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 2. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

The visible page comes first

Rendering is a scheduling problem

3.1 The visible page comes first

A document with a thousand pages does not require a thousand page images. The immediate workload is a handful of visible tiles. Requests near the viewport have priority; speculative work is useful only when it does not delay those tiles. Finch uses bounded queues to make this tradeoff explicit.

Every render request names the document generation that created it. A later scroll, zoom, or file change can make that request obsolete. Cancellation saves work where the engine offers a pause point. Otherwise the completed result is discarded before it can replace newer content.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 3. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Budgets make caches predictable

Count bytes rather than entries

4.1 Budgets make caches predictable

A cache with ten entries can be tiny or enormous. A byte budget expresses the resource that actually matters. Tile buffers, copied upload buffers, and retained textures each have a lifetime, so memory accounting follows ownership rather than the number of pages visited.

Eviction should preserve the current view before preserving distant history. A useful experiment scrolls forward through a document, returns to a nearby page, and then jumps far away. This reveals both locality and whether old generations continue to occupy the cache.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 4. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Wide figures and mixed pages

One document can contain several geometries

5.1 Wide figures and mixed pages

This landscape page is intentionally wider than the preceding pages. Mixed page sizes expose assumptions that disappear in a uniform document. The layout should use the dimensions of each page while retaining a stable reading anchor as new dimensions become available.

A wide figure can be inspected at fit-to-width or at a deliberate scale. Horizontal position matters in the second case. A user should not lose the relevant part of a diagram simply because a neighboring portrait page has a different width.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 5. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Text is more than an image

Selection needs the same coordinate system

6.1 Text is more than an image

A rendered tile tells the eye where letters are. Text extraction tells the application what those letters mean. Selection joins these representations using page-space bounds. The same conversion should support highlights, link hit testing, and navigation targets.

A rotated page is a particularly useful check. A text box that appears correct on an unrotated page can drift after rotation if only one side of the transformation is adjusted. Test points at the corners as well as near the center of the page.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 6. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Search without blocking reading

Results can arrive incrementally

7.1 Search without blocking reading

Finch appears throughout this original fixture so that document search has several meaningful matches. A search request is a visitor to the PDF worker, not its permanent owner. Visible rendering can take precedence between pages while partial matches become available.

Changing the query invalidates earlier results. Keep the query generation with each response, just as rendering keeps the document generation. This avoids a familiar interface error: a result list that silently combines two different user requests.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 7. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Navigation is a local operation

Bookmarks, links, and page numbers agree

8.1 Navigation is a local operation

The bookmark tree in this fixture contains one entry for every page. The first page also links to the wide-page section. These are internal destinations and should move the reading position without reopening the file or discarding the current document identity.

External URI links are a separate kind of destination. This fixture points to example.org, a deliberately illustrative address. Displaying a link and opening a link are separate actions; the latter follows an explicit reader gesture.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 8. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

A smaller sheet

Compact pages still need comfortable margins

9.1 A smaller sheet

Finch displays this page on a smaller sheet. The text remains real selectable content, so zooming can be checked without confusing bitmap sharpness with page layout. The footer and heading stay inside the page bounds rather than relying on the dimensions of the first page.

Page-size changes are also useful restoration tests. Stop near the bottom of this sheet, close the application, then reopen it with a different window width. The same page-space anchor should remain the reference point.

Figure 9. A conceptual reading pipeline; no measured values are implied.

```
state = page + anchor + zoom
```

A library of references

The original file remains the source

10.1 A library of references

A lightweight library records a reference to a local PDF. Importing the reference need not duplicate the file. Titles, tags, favorites, and recent reading information help a person return to a paper without converting the library into a second filesystem.

Files do move. A missing reference should remain visible and offer a way to reconnect it. Removing a library record is distinct from deleting the original PDF. This distinction belongs in the application behavior, not merely in documentation.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 10. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Saving small pieces of state

Durability should not interrupt attention

11.1 Saving small pieces of state

Scrolling can change the reading position dozens of times per second. Writing every change is unnecessary. A short debounce coalesces nearby changes, while a maximum interval prevents continuous interaction from postponing persistence indefinitely.

A normal exit flushes the final state. A forced exit is different: the last durable snapshot is the recovery point. Atomic replacement ensures that recovery sees either the prior complete snapshot or the new complete snapshot, rather than a partially written record.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 11. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Plugins at the edge

A command earns its process

12.1 Plugins at the edge

A translation command or a note-taking command can extend Finch without changing the PDF engine. The host supplies a small context: document information, page index, and selected text. A plain-text result is sufficient for the first executable plugin protocol.

Trusted native plugins run only after an explicit command. The process boundary helps contain ordinary crashes, but it is not a security sandbox. A protocol needs version negotiation, bounded messages, deadlines, and a clear cleanup path when a child stops responding.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 12. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

A second portrait format

Letter-sized paper joins the sequence

13.1 A second portrait format

This page uses US Letter dimensions. Together with the wide and compact pages it tests layout across several common geometries. A page index always identifies the same content, regardless of how many pixels are needed to display that content at the current scale.

The diagram below is a conceptual pipeline rather than a measured performance chart. It shows why the data crossing module boundaries should be simple: document identifiers, page coordinates, requests, and results. Native engine handles stay inside their owning worker.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 13. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Measure what the reader sees

A milestone is not a whole experience

14.1 Measure what the reader sees

A window-open timestamp measures one stage. It does not prove that the recovered page is legible. Tile completion is another stage, and actual presentation is another. Report these separately rather than using a convenient internal event as a substitute for every user-visible outcome.

Warm startup and cold startup answer different questions. Repeated runs naturally warm filesystem and driver caches. A cold sample needs an explicit environmental procedure. Do not label the first run of a script as a cold boot unless those conditions were controlled.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 14. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

Memory and idle time

Independent observations catch blind spots

15.1 Memory and idle time

A tile-cache counter describes application-owned buffers. It does not include every allocation in a PDF parser, a renderer, or the operating system. Process private bytes and working set are useful independent observations, while GPU memory needs a separate measurement method.

Idle CPU should be sampled during a quiet interval without automation that repeatedly moves the pointer, captures a screenshot, or changes the window. Record the duration and normalization convention. A small number without its method is difficult to compare.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 15. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```

A reproducible reading exercise

The fixture is a tool, not a benchmark result

16.1 A reproducible reading exercise

This document is original authored content for Finch development. Its sixteen pages provide selectable text, headings, simple equations, vector diagrams, bookmarks, internal navigation, an external URI, and mixed page dimensions. It contains no third-party paper or remote media.

The equations and diagrams are conceptual. They do not report measured results or make claims about completed performance targets. Use the fixture to make tests repeatable, and keep the actual acceptance evidence in a separate report with commands, dates, hardware, and limitations.

Reading as a sequence

A useful reading session combines small actions: locate a passage, compare a figure, search a term, and return to the argument. Each action has a different data need. The interface can remain compact when the work behind those actions is separated into modules with explicit ownership.

For this page, try selecting the word **Finch**, changing the zoom, and then following a bookmark. Reopen the application after closing it. A repeatable fixture makes it possible to compare the same interaction before and after an implementation change.

Design note. The statements here describe desired behavior and test methods. They are not evidence that latency, memory, or frame-rate budgets have already been met.

Figure 16. A conceptual reading pipeline; no measured values are implied.

```
work(t) = visible(t) + bounded_prefetch(t)
```